Cheatsheet — Bases de Datos (MySQL + PHP)

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** MySQL / MariaDB + PHP con `mysqli` (estilo procedural, como en la teoría).

Índice

1. Conceptos que siempre preguntan
2. Diseño: del diagrama ER a las tablas
3. SQL — DDL (crear/modificar estructura)
4. SQL — DML (insertar/actualizar/borrar)
5. SQL — SELECT (consultas)
6. JOINs
7. Subconsultas
8. Funciones útiles
9. Vistas, índices y EXPLAIN
10. Transacciones y motores
11. Usuarios y permisos
12. PHP + MySQLi
13. Seguridad
14. phpMyAdmin y consola
15. Plantillas para el parcial
16. Ejercicios resueltos
17. Apéndice: proyecto de convenios
18. Errores típicos y checklist final

1. Conceptos que siempre preguntan

Concepto	Definición corta	Ejemplo
Base de datos	Conjunto organizado de datos relacionados.	`pow`
SGBD	Software que administra la base: MySQL, MariaDB, PostgreSQL.	MySQL
Tabla / relación	Estructura de filas y columnas sobre una entidad.	`noticias`
Fila / registro / tupla	Un elemento concreto de la tabla.	Una noticia
Columna / campo / atributo	Propiedad de la entidad, con un tipo de dato.	`titulo VARCHAR(150)`
Clave primaria (PK)	Columna(s) que identifican de forma única cada fila. No admite `NULL`.	`id`
Clave foránea (FK)	Columna que referencia la PK de otra tabla. Garantiza integridad referencial.	`categoria_id → categorias.id`
Clave candidata / UNIQUE	Columna(s) que podrían ser PK; no se repiten.	`usuario`
NULL	Ausencia de valor. **No** es `0` ni `''`. Se compara con `IS NULL`.	`imagen IS NULL`
Integridad referencial	No puede existir una FK que apunte a una fila inexistente.	FK con `REFERENCES`
Índice	Estructura auxiliar que acelera búsquedas por una columna.	`INDEX (fecha)`
Vista	Consulta guardada que se usa como si fuera una tabla.	`v_noticias`

Relaciones

Relación	Cómo se implementa	Ejemplo
1:1	FK con `UNIQUE` en uno de los lados.	`usuarios` ↔ `perfiles`
1:N	FK en la tabla del lado "muchos".	`categorias` 1 → N `noticias`
N:M	Tabla intermedia con dos FK y PK compuesta.	`noticias` ↔ `etiquetas`

Normalización (respuesta corta de parcial)

- **1FN:** todos los valores son atómicos; no hay listas ni campos repetidos.
- **2FN:** cada atributo depende de **toda** la PK (importante en PK compuestas).

- **3FN:** ningún atributo depende de otro atributo que no sea la PK (sin dependencias transitivas).
- **Objetivo:** evitar redundancia y anomalías de inserción, actualización y borrado.
- **Desnormalizar** a veces se hace por rendimiento, pero se debe justificar.

> **Regla práctica:** si un dato se repite en muchas filas (por ejemplo, el nombre de la categoría), conviene sacarlo a otra tabla y relacionarlo con FK.

---

2. Diseño: del diagrama ER a las tablas

Pasos

1. Identificar **entidades** (sustantivos): noticia, categoría, usuario, etiqueta.
2. Identificar **atributos** y elegir la PK de cada entidad.
3. Identificar **relaciones** y su cardinalidad.
4. Resolver N:M con tabla intermedia.
5. Definir tipos, `NOT NULL`, `UNIQUE`, `DEFAULT` y FK.
6. Revisar normalización.
7. Probar con datos de ejemplo y consultas reales.

Ejemplo de esquema

```
CREATE DATABASE IF NOT EXISTS pow
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE pow;

CREATE TABLE categorias (
  id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE usuarios (
  id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  usuario    VARCHAR(50)  NOT NULL UNIQUE,
  clave_hash VARCHAR(255) NOT NULL,
  rol        ENUM('admin','editor','lector') NOT NULL DEFAULT 'lector',
  creado_en  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE noticias (
  id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  titulo       VARCHAR(150) NOT NULL,
  copete       VARCHAR(255),
  cuerpo       TEXT NOT NULL,
  categoria_id INT UNSIGNED NOT NULL,
  autor_id     INT UNSIGNED,
  fecha        DATE NOT NULL,
  imagen       VARCHAR(255),
  CONSTRAINT fk_noticias_categoria
    FOREIGN KEY (categoria_id) REFERENCES categorias(id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_noticias_autor
    FOREIGN KEY (autor_id) REFERENCES usuarios(id)
    ON DELETE SET NULL
) ENGINE=InnoDB;
```

Tabla intermedia N:M

```
CREATE TABLE etiquetas (
  id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE noticia_etiqueta (
  noticia_id  INT UNSIGNED NOT NULL,
  etiqueta_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (noticia_id, etiqueta_id),
  CONSTRAINT fk_ne_noticia
    FOREIGN KEY (noticia_id) REFERENCES noticias(id) ON DELETE CASCADE,
  CONSTRAINT fk_ne_etiqueta
```

```
    FOREIGN KEY (etiqueta_id) REFERENCES etiquetas(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

> **Ojo:** la PK compuesta `(noticia_id, etiqueta_id)` impide cargar dos veces la misma relación.

---

3. SQL — DDL (crear/modificar estructura)

Crear, usar y borrar

```
CREATE DATABASE pow;
USE pow;
DROP DATABASE IF EXISTS pow;

CREATE TABLE IF NOT EXISTS alumnos (
  id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(80) NOT NULL,
  email     VARCHAR(120) UNIQUE,
  edad      TINYINT UNSIGNED,
  activo    BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

DROP TABLE IF EXISTS alumnos;
TRUNCATE TABLE alumnos;   -- vacía y reinicia AUTO_INCREMENT (no se puede deshacer)
```

Tipos de datos más usados

Tipo	Uso	Ejemplo
`TINYINT`, `SMALLINT`, `INT`, `BIGINT`	Enteros de distinto rango.	`edad TINYINT UNSIGNED`
`DECIMAL(p,s)`	Dinero / valores exactos.	`precio DECIMAL(10,2)`
`FLOAT`, `DOUBLE`	Cálculos aproximados.	`promedio DOUBLE`
`CHAR(n)`	Texto de longitud fija.	`codigo CHAR(3)`
`VARCHAR(n)`	Texto variable hasta `n`.	`titulo VARCHAR(150)`
`TEXT`, `LONGTEXT`	Texto largo.	`cuerpo TEXT`
`ENUM('a','b')`	Uno de varios valores.	`estado ENUM('activo','inactivo')`
`DATE`	Fecha `YYYY-MM-DD`.	`fecha DATE`
`TIME`	Hora `HH:MM:SS`.	`hora TIME`
`DATETIME`	Fecha y hora.	`publicado_en DATETIME`
`TIMESTAMP`	Fecha/hora con zona y actualización automática.	`actualizado_en TIMESTAMP`
`BOOLEAN`	Alias de `TINYINT(1)`: `0` / `1`.	`activo BOOLEAN`
`JSON`	Documento JSON.	`datos JSON`
`BLOB`	Datos binarios (imagen, PDF).	`archivo BLOB`

Restricciones (`constraints`)

Restricción	Qué hace
`PRIMARY KEY`	Identifica la fila. Única y `NOT NULL`.
`FOREIGN KEY ... REFERENCES`	Valida que el valor exista en la tabla referenciada.
`UNIQUE`	No permite valores repetidos (admite varios `NULL`).
`NOT NULL`	Obliga a cargar un valor.
`DEFAULT`	Valor si no se envía nada.
`CHECK`	Valida una condición (MySQL 8.0.16+).
`AUTO_INCREMENT`	Genera el siguiente entero automáticamente.

```
CHECK (edad >= 0 AND edad <= 120)
CHECK (fecha_vencimiento >= fecha_firma)
```

Modificar una tabla

```
ALTER TABLE noticias ADD COLUMN destacada BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE noticias MODIFY COLUMN titulo VARCHAR(200) NOT NULL;
ALTER TABLE noticias CHANGE COLUMN copete resumen VARCHAR(255);
ALTER TABLE noticias RENAME COLUMN resumen TO copete;  -- MySQL 8+
ALTER TABLE noticias DROP COLUMN imagen;

ALTER TABLE noticias ADD INDEX idx_fecha (fecha);
ALTER TABLE noticias DROP INDEX idx_fecha;

ALTER TABLE noticias DROP FOREIGN KEY fk_noticias_categoria;
ALTER TABLE noticias ADD CONSTRAINT fk_noticias_categoria
  FOREIGN KEY (categoria_id) REFERENCES categorias(id);
```

Acciones referenciales

Acción	Qué pasa al borrar/actualizar el padre
`RESTRICT` / `NO ACTION`	Impide la operación si hay hijos. Es la más segura.
`CASCADE`	Borra/actualiza también los hijos.
`SET NULL`	Pone `NULL` en la FK del hijo (la columna debe admitir `NULL`).
`SET DEFAULT`	Usa el valor por defecto (poco usado en InnoDB).

Ver estructura

```
SHOW DATABASES;
SHOW TABLES;
DESCRIBE noticias;            -- o DESC noticias;
SHOW CREATE TABLE noticias;
SHOW COLUMNS FROM noticias;
SHOW INDEX FROM noticias;
```

---

4. SQL — DML (insertar/actualizar/borrar)

INSERT

```
INSERT INTO categorias (nombre)
VALUES ('promociones'), ('ofertas'), ('costas');

INSERT INTO usuarios (usuario, clave_hash, rol)
VALUES ('guillermo', '$2y$10$...', 'admin');

-- Insertar a partir de otra consulta
INSERT INTO noticias (titulo, cuerpo, categoria_id, fecha)
SELECT 'Nueva oferta', 'Texto...', id, CURDATE()
FROM categorias
WHERE nombre = 'ofertas';
```

UPDATE

```
UPDATE noticias
SET titulo = 'Título corregido', categoria_id = 2
WHERE id = 5;

-- Actualizar usando JOIN
UPDATE noticias n
JOIN categorias c ON c.id = n.categoria_id
SET n.destacada = TRUE
WHERE c.nombre = 'ofertas';
```

> **Siempre** probar el `WHERE` con un `SELECT` antes de un `UPDATE` o `DELETE`.

DELETE

```
DELETE FROM noticias WHERE id = 5;

-- Borrar solo si no tiene hijos (si la FK es RESTRICT, fallará)
DELETE FROM categorias WHERE id = 3;
```

```
-- Vaciar tabla (más rápido, reinicia AUTO_INCREMENT)
TRUNCATE TABLE noticias;
```

`ON DUPLICATE KEY UPDATE`

```
INSERT INTO categorias (id, nombre)
VALUES (1, 'promociones')
ON DUPLICATE KEY UPDATE nombre = VALUES(nombre);
```

Diferencias rápidas

Operación	¿Se puede deshacer?	¿Reinicia AUTO_INCREMENT?
`DELETE`	Sí, con transacción.	No.
`TRUNCATE`	No (es DDL).	Sí.
`DROP`	No.	—

---

5. SQL — SELECT (consultas)

Sintaxis y orden de ejecución

```
SELECT [DISTINCT] columnas
FROM tabla
[JOIN otra_tabla ON condición]
WHERE condición
GROUP BY columnas
HAVING condición_de_grupo
ORDER BY columnas [ASC|DESC]
LIMIT cantidad OFFSET desplazamiento;
```

Orden lógico de ejecución (muy preguntado):

```
FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
```

> Por eso no se puede usar un alias del `SELECT` en el `WHERE` (sí en `ORDER BY` y `HAVING`).

WHERE: operadores

Operador	Ejemplo
`=`, `<>` / `!=`, `<`, `>`, `<=`, `>=`	`edad >= 18`
`AND`, `OR`, `NOT`	`activo = 1 AND edad >= 18`
`BETWEEN a AND b`	`fecha BETWEEN '2026-01-01' AND '2026-12-31'`
`IN (...)`	`categoria_id IN (1, 2, 3)`
`NOT IN (...)`	`id NOT IN (SELECT ...)`
`LIKE`	`titulo LIKE '%promo%'`
`IS NULL` / `IS NOT NULL`	`imagen IS NULL`
`EXISTS`	`EXISTS (SELECT 1 FROM ...)`

Comodines de `LIKE`:

Comodín	Significado
`%`	Cualquier cantidad de caracteres (incluido cero).
`_`	Exactamente un carácter.
`[abc]`	Solo en SQL Server; en MySQL usar `REGEXP`.

```
SELECT * FROM noticias WHERE titulo LIKE 'Promo%';   -- empieza con Promo
SELECT * FROM noticias WHERE titulo LIKE '%casa%';   -- contiene casa
SELECT * FROM noticias WHERE titulo REGEXP '^[0-9]+'; -- empieza con números
```

ORDER BY y LIMIT

```
SELECT titulo, fecha
FROM noticias
```

```
ORDER BY fecha DESC, titulo ASC
LIMIT 10 OFFSET 0;       -- 10 registros desde el primero

SELECT * FROM noticias ORDER BY id DESC LIMIT 5;  -- últimos 5
```

DISTINCT

```
SELECT DISTINCT categoria_id FROM noticias;
SELECT DISTINCT categoria_id, autor_id FROM noticias;
```

Funciones de agregación

Función	Qué hace
`COUNT(*)`	Cuenta filas (incluye `NULL`).
`COUNT(columna)`	Cuenta valores no `NULL`.
`SUM(columna)`	Suma. Devuelve `NULL` si no hay filas.
`AVG(columna)`	Promedio.
`MIN(columna)` / `MAX(columna)`	Mínimo / máximo.
`GROUP_CONCAT(columna SEPARATOR ', ')`	Une valores de un grupo en un string.

GROUP BY y HAVING

```
SELECT c.nombre AS categoria, COUNT(*) AS total
FROM noticias n
JOIN categorias c ON c.id = n.categoria_id
GROUP BY c.id, c.nombre
HAVING COUNT(*) >= 2
ORDER BY total DESC;
```

> **Regla:** toda columna del `SELECT` que no esté dentro de una función de agregación debe aparecer en el `GROUP BY` (si está activo `ONLY_FULL_GROUP_BY`).

CASE e IF

```
SELECT titulo,
       CASE
         WHEN categoria_id = 1 THEN 'Promoción'
         WHEN categoria_id = 2 THEN 'Oferta'
         ELSE 'Otra'
       END AS tipo
FROM noticias;

SELECT titulo, IF(destacada = 1, 'Sí', 'No') AS destacada
FROM noticias;
```

Valores únicos y conteos rápidos

```
SELECT COUNT(*) FROM noticias;
SELECT COUNT(DISTINCT categoria_id) FROM noticias;
SELECT MAX(fecha), MIN(fecha) FROM noticias;
```

---

6. JOINs

Tipos

JOIN	Devuelve
`INNER JOIN`	Solo filas con coincidencia en ambas tablas.
`LEFT JOIN`	Todas las de la izquierda + coincidencias de la derecha (`NULL` si no hay).
`RIGHT JOIN`	Todas las de la derecha + coincidencias de la izquierda.
`CROSS JOIN`	Producto cartesiano (todas las combinaciones).
`SELF JOIN`	La tabla se une consigo misma con alias.

```
-- INNER: noticias con su categoría
SELECT n.id, n.titulo, c.nombre AS categoria
FROM noticias n
INNER JOIN categorias c ON c.id = n.categoria_id;

-- LEFT: todas las categorías, tengan o no noticias
SELECT c.nombre, COUNT(n.id) AS total
FROM categorias c
LEFT JOIN noticias n ON n.categoria_id = c.id
GROUP BY c.id, c.nombre;

-- Anti-join: categorías sin noticias
SELECT c.*
FROM categorias c
LEFT JOIN noticias n ON n.categoria_id = c.id
WHERE n.id IS NULL;

-- SELF JOIN: empleados y su jefe
SELECT e.nombre AS empleado, j.nombre AS jefe
FROM empleados e
LEFT JOIN empleados j ON j.id = e.jefe_id;
```

N:M

```
SELECT n.titulo, e.nombre AS etiqueta
FROM noticias n
JOIN noticia_etiqueta ne ON ne.noticia_id = n.id
JOIN etiquetas e       ON e.id = ne.etiqueta_id
WHERE n.id = 1;
```

UNION

```
SELECT titulo AS texto FROM noticias
UNION
SELECT nombre AS texto FROM categorias;
```

- `UNION` elimina duplicados y ordena; `UNION ALL` los conserva y es más rápido.
- Todas las consultas deben tener la misma cantidad y tipo de columnas.

FULL OUTER JOIN (no existe en MySQL)

```
SELECT ... FROM a LEFT JOIN b ON ...
UNION
SELECT ... FROM a RIGHT JOIN b ON ...;
```

---

7. Subconsultas

```
-- IN
SELECT * FROM noticias
WHERE categoria_id IN (
  SELECT id FROM categorias WHERE nombre = 'ofertas'
);

-- EXISTS (suele ser más eficiente para correlacionadas)
SELECT n.*
FROM noticias n
WHERE EXISTS (
  SELECT 1 FROM comentarios c WHERE c.noticia_id = n.id
);

-- Subconsulta escalar en SELECT
SELECT n.titulo,
       (SELECT COUNT(*) FROM comentarios c WHERE c.noticia_id = n.id) AS comentarios
FROM noticias n;

-- Tabla derivada (subconsulta en FROM)
SELECT *
FROM (
  SELECT categoria_id, COUNT(*) AS total
  FROM noticias
  GROUP BY categoria_id
) t
```

```
WHERE t.total > 5;

-- Comparar con ANY / ALL
SELECT * FROM alumnos
WHERE nota > ALL (SELECT nota FROM alumnos WHERE carrera = 'Sistemas');
```

> **Cuidado con `NOT IN`:** si la subconsulta devuelve algún `NULL`, el resultado puede quedar vacío. Usar `NOT EXISTS` o filtrar con `IS NOT NULL`.

---

8. Funciones útiles

Strings

Función	Ejemplo	Resultado
`CONCAT(a,b)`	`CONCAT('Hola',' ','mundo')`	`Hola mundo`
`CONCAT_WS(sep,...)`	`CONCAT_WS('-','2026','10','25')`	`2026-10-25`
`LENGTH(s)` / `CHAR_LENGTH(s)`	`CHAR_LENGTH('ñ')`	Bytes vs caracteres
`UPPER(s)` / `LOWER(s)`	`UPPER('hola')`	`HOLA`
`TRIM(s)` / `LTRIM` / `RTRIM`	`TRIM(' hola ')`	`hola`
`SUBSTRING(s,pos,len)`	`SUBSTRING('abcdef',2,3)`	`bcd`
`LEFT(s,n)` / `RIGHT(s,n)`	`LEFT('abcdef',3)`	`abc`
`REPLACE(s,a,b)`	`REPLACE('a-b','-','/')`	`a/b`
`LOCATE(sub,s)`	`LOCATE('c','abc')`	`3` (0 si no está)
`LPAD(s,n,c)` / `RPAD`	`LPAD('7',3,'0')`	`007`
`FORMAT(n,d)`	`FORMAT(1234.5,2)`	`1,234.50`

Fechas

Función	Ejemplo	Resultado
`NOW()`	Fecha y hora actual.	`2026-10-25 10:30:00`
`CURDATE()` / `CURTIME()`	Solo fecha / solo hora.	`2026-10-25`
`DATE(col)`	Extrae la fecha de un `DATETIME`.	`2026-10-25`
`YEAR(col)`, `MONTH(col)`, `DAY(col)`	Partes de una fecha.	`2026`, `10`, `25`
`DATE_FORMAT(col,'%d/%m/%Y')`	Formatea.	`25/10/2026`
`STR_TO_DATE('25/10/2026','%d/%m/%Y')`	Convierte texto a fecha.	`2026-10-25`
`DATEDIFF(a,b)`	Días entre dos fechas.	`a - b`
`TIMESTAMPDIFF(YEAR,nac,CURDATE())`	Años cumplidos (edad).	`30`
`DATE_ADD(col, INTERVAL 30 DAY)`	Suma intervalo.	Fecha + 30 días
`DATE_SUB(col, INTERVAL 1 MONTH)`	Resta intervalo.	Fecha - 1 mes
`LAST_DAY(col)`	Último día del mes.	`2026-10-31`

Formatos de `DATE_FORMAT`:

Código	Significado	Código	Significado
`%Y`	Año 4 dígitos	`%m`	Mes 01-12
`%y`	Año 2 dígitos	`%c`	Mes 1-12
`%d`	Día 01-31	`%e`	Día 1-31
`%H`	Hora 00-23	`%i`	Minutos 00-59
`%s`	Segundos 00-59	`%W`	Nombre del día
`%M`	Nombre del mes	`%j`	Día del año

Números y NULL

```
SELECT ROUND(12.345, 2);        -- 12.35
SELECT TRUNCATE(12.345, 2);     -- 12.34
SELECT CEIL(12.1), FLOOR(12.9); -- 13, 12
SELECT MOD(10, 3);              -- 1
SELECT ABS(-5);                 -- 5

SELECT IFNULL(imagen, 'sin-imagen') FROM noticias;
SELECT COALESCE(copete, titulo, 'Sin título') FROM noticias;
SELECT NULLIF(a, b);            -- NULL si a = b
```

> `COUNT(*)` cuenta filas; `COUNT(columna)` ignora `NULL`. `SUM` y `AVG` también ignoran `NULL`.

---

9. Vistas, índices y EXPLAIN

Vistas

```
CREATE OR REPLACE VIEW v_noticias_categoria AS
SELECT n.id, n.titulo, c.nombre AS categoria, n.fecha
FROM noticias n
JOIN categorias c ON c.id = n.categoria_id;

SELECT * FROM v_noticias_categoria WHERE categoria = 'ofertas';
DROP VIEW IF EXISTS v_noticias_categoria;
```

- Ventajas: simplifican consultas, ocultan columnas sensibles, reutilizan lógica.
- No todas las vistas son actualizables (depende de joins, agregaciones, etc.).

Índices

```
CREATE INDEX idx_noticias_fecha ON noticias (fecha);
CREATE UNIQUE INDEX uq_usuarios_usuario ON usuarios (usuario);
CREATE INDEX idx_noticias_cat_fecha ON noticias (categoria_id, fecha);

SHOW INDEX FROM noticias;
DROP INDEX idx_noticias_fecha ON noticias;
```

- Aceleran `WHERE`, `JOIN`, `ORDER BY` y `GROUP BY`.
- Demasiados índices hacen más lentos los `INSERT` / `UPDATE` / `DELETE`.
- En un índice compuesto, el orden importa: sirve para prefijos (`categoria_id`, luego `fecha`).
- Las FK conviene indexarlas (MySQL lo hace automáticamente).

EXPLAIN

```
EXPLAIN SELECT * FROM noticias WHERE categoria_id = 1;
EXPLAIN ANALYZE SELECT ...;  -- MySQL 8.0.18+
```

Mirar: `type` (`ALL` es malo), `key` (índice usado), `rows` (filas estimadas), `Extra`.

---

10. Transacciones y motores

ACID

Propiedad	Significado
Atomicidad	Todo o nada.
Consistencia	Se respetan las restricciones.
Isolation	Las transacciones no interfieren entre sí.
Durabilidad	Lo confirmado queda guardado.

```
START TRANSACTION;   -- o BEGIN;

UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
```

```
COMMIT;             -- confirma
-- ROLLBACK;        -- deshace todo
```

Motores

Motor	Transacciones	FK	Uso
InnoDB	Sí	Sí	Recomendado por defecto.
MyISAM	No	No	Legacy; evitar.

```
SHOW ENGINES;
ALTER TABLE noticias ENGINE = InnoDB;
```

Niveles de aislamiento

```
SELECT @@transaction_isolation;
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Por defecto InnoDB usa `REPEATABLE READ`. Para bloquear filas:

```
SELECT * FROM cuentas WHERE id = 1 FOR UPDATE;
```

> Los `CREATE`, `ALTER`, `DROP` y `TRUNCATE` son DDL: no se pueden revertir con `ROLLBACK` en MySQL.

---

11. Usuarios y permisos

```
CREATE USER 'app_pow'@'localhost' IDENTIFIED BY 'ClaveSegura123!';

GRANT SELECT, INSERT, UPDATE, DELETE ON pow.* TO 'app_pow'@'localhost';
GRANT SELECT ON pow.noticias TO 'app_pow'@'localhost';

FLUSH PRIVILEGES;  -- necesario solo si se modifican tablas del sistema a mano

SHOW GRANTS FOR 'app_pow'@'localhost';
REVOKE DELETE ON pow.* FROM 'app_pow'@'localhost';

ALTER USER 'app_pow'@'localhost' IDENTIFIED BY 'NuevaClave123!';
DROP USER 'app_pow'@'localhost';
```

Roles (MySQL 8+)

```
CREATE ROLE 'lector';
GRANT SELECT ON pow.* TO 'lector';
GRANT 'lector' TO 'app_pow'@'localhost';
SET DEFAULT ROLE 'lector' TO 'app_pow'@'localhost';
```

> **Principio de mínimo privilegio:** la aplicación web no debe conectarse como `root`. Crear un usuario solo con los permisos que necesita.

Backup y restore

```
mysqldump -u root -p pow > backup_pow.sql
mysql -u root -p pow < backup_pow.sql
```

---

12. PHP + MySQLi

Conexión

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$conexion = mysqli_connect('localhost', 'root', '', 'pow');
mysqli_set_charset($conexion, 'utf8mb4');

if (!$conexion) {
    die('Error de conexión: ' . mysqli_connect_error());
}
```

> En producción no mostrar el error al usuario: registrarlo en un log y mostrar un mensaje genérico.

Consulta simple (sin datos del usuario)

```
$resultado = mysqli_query($conexion, "SELECT id, titulo, fecha FROM noticias ORDER BY fecha DESC");

$total = mysqli_num_rows($resultado);

while ($fila = mysqli_fetch_assoc($resultado)) {
    echo '<h3>' . htmlspecialchars($fila['titulo'], ENT_QUOTES, 'UTF-8') . '</h3>';
    echo '<p>' . htmlspecialchars($fila['fecha'], ENT_QUOTES, 'UTF-8') . '</p>';
}

mysqli_free_result($resultado);
mysqli_close($conexion);
```

`mysqli_fetch_assoc()` devuelve un array asociativo o `null` al terminar. También existen `mysqli_fetch_array($res, MYSQLI_ASSOC)` y `mysqli_fetch_object()`.

Consulta con parámetros (prepared statement)

```
$categoria_id = (int) ($_GET['categoria'] ?? 0);

$sql = "SELECT id, titulo, fecha
        FROM noticias
        WHERE categoria_id = ?
        ORDER BY fecha DESC";

$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'i', $categoria_id);
mysqli_stmt_execute($stmt);

$resultado = mysqli_stmt_get_result($stmt);

while ($fila = mysqli_fetch_assoc($resultado)) {
    echo htmlspecialchars($fila['titulo'], ENT_QUOTES, 'UTF-8');
}

mysqli_stmt_close($stmt);
```

INSERT

```
$titulo      = trim($_POST['titulo'] ?? '');
$cuerpo      = trim($_POST['cuerpo'] ?? '');
$categoria_id = (int) ($_POST['categoria_id'] ?? 0);
$fecha       = $_POST['fecha'] ?? date('Y-m-d');

$sql = "INSERT INTO noticias (titulo, cuerpo, categoria_id, fecha)
        VALUES (?, ?, ?, ?)";

$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'ssis', $titulo, $cuerpo, $categoria_id, $fecha);

if (mysqli_stmt_execute($stmt)) {
    $nuevo_id = mysqli_insert_id($conexion);
    echo "Noticia creada con ID: $nuevo_id";
}

mysqli_stmt_close($stmt);
```

UPDATE / DELETE

```
$sql = "UPDATE noticias SET titulo = ?, categoria_id = ? WHERE id = ?";
$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'sii', $titulo, $categoria_id, $id);
mysqli_stmt_execute($stmt);

echo "Filas afectadas: " . mysqli_stmt_affected_rows($stmt);
mysqli_stmt_close($stmt);

$stmt = mysqli_prepare($conexion, "DELETE FROM noticias WHERE id = ?");
mysqli_stmt_bind_param($stmt, 'i', $id);
mysqli_stmt_execute($stmt);
```

Tipos de `bind_param`

Letra	Tipo
`i`	Entero
`d`	Decimal / `double`
`s`	String
`b`	Binario (blob)

El primer argumento es una cadena con una letra por cada `?`, en orden.

Paginación

```
$por_pagina = 5;
$pagina     = max(1, (int) ($_GET['pagina'] ?? 1));
$offset     = ($pagina - 1) * $por_pagina;

$sql = "SELECT id, titulo FROM noticias ORDER BY fecha DESC LIMIT ? OFFSET ?";
$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'ii', $por_pagina, $offset);
mysqli_stmt_execute($stmt);
$resultado = mysqli_stmt_get_result($stmt);

while ($fila = mysqli_fetch_assoc($resultado)) {
    echo htmlspecialchars($fila['titulo'], ENT_QUOTES, 'UTF-8') . '<br>';
}

// Enlaces
if ($pagina > 1) {
    echo '<a href="?pagina=' . ($pagina - 1) . '">Anterior</a>';
}
echo '<a href="?pagina=' . ($pagina + 1) . '">Siguiente</a>';
```

Login con sesión

```
session_start();

$usuario = trim($_POST['usuario'] ?? '');
$clave   = $_POST['clave'] ?? '';

$sql = "SELECT id, usuario, clave_hash, rol
        FROM usuarios
        WHERE usuario = ?
        LIMIT 1";

$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 's', $usuario);
mysqli_stmt_execute($stmt);
$resultado = mysqli_stmt_get_result($stmt);
$u = mysqli_fetch_assoc($resultado);

if ($u && password_verify($clave, $u['clave_hash'])) {
    session_regenerate_id(true);
    $_SESSION['usuario_id'] = $u['id'];
    $_SESSION['usuario']    = $u['usuario'];
    $_SESSION['rol']        = $u['rol'];
    header('Location: menu.php');
    exit;
} else {
    echo 'Usuario o clave incorrectos';
}
```

Transacción en PHP

```
mysqli_begin_transaction($conexion);

try {
    mysqli_query($conexion, "UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1");
    mysqli_query($conexion, "UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2");
    mysqli_commit($conexion);
} catch (Throwable $e) {
    mysqli_rollback($conexion);
```

```
    throw $e;
}
```

Funciones `mysqli` que conviene recordar

Función	Para qué
`mysqli_connect()`	Conectar.
`mysqli_select_db()`	Seleccionar base (o pasar el 4.º argumento).
`mysqli_query()`	Ejecutar SQL simple.
`mysqli_prepare()`	Preparar consulta con `?`.
`mysqli_stmt_bind_param()`	Vincular parámetros.
`mysqli_stmt_execute()`	Ejecutar preparada.
`mysqli_stmt_get_result()`	Obtener resultado (mysqlnd).
`mysqli_fetch_assoc()`	Fila como array asociativo.
`mysqli_num_rows()`	Cantidad de filas de un `SELECT`.
`mysqli_affected_rows()`	Filas afectadas por `INSERT` / `UPDATE` / `DELETE`.
`mysqli_insert_id()`	Último `AUTO_INCREMENT`.
`mysqli_error()` / `mysqli_stmt_error()`	Mensaje de error (no mostrar al usuario).
`mysqli_close()`	Cerrar conexión.

13. Seguridad

Inyección SQL

```
// ❌ MAL: concatenar datos del usuario
$sql = "SELECT * FROM usuarios
        WHERE usuario = '$usuario' AND clave = '$clave'";

// Entrada maliciosa:
// usuario: ' OR '1'='1' --
// Resultado: SELECT * FROM usuarios WHERE usuario = '' OR '1'='1' --' AND ...
```

```
// ✅ BIEN: prepared statement
$sql = "SELECT id, clave_hash FROM usuarios WHERE usuario = ? LIMIT 1";
$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 's', $usuario);
mysqli_stmt_execute($stmt);
```

Reglas de oro

1. **Nunca** concatenar datos del usuario en SQL. Usar `?` + `bind_param`.
2. Los nombres de tablas/columnas **no** se pueden parametrizar: validarlos contra una lista blanca.
3. Escapar la salida con `htmlspecialchars($texto, ENT_QUOTES, 'UTF-8')` para evitar XSS.
4. Guardar claves con `password_hash($clave, PASSWORD_DEFAULT)` y verificar con `password_verify()`.
5. No usar `md5()` ni `sha1()` para contraseñas.
6. Validar tipos: `(int)`, `filter_var()`, `DateTime::createFromFormat()`.
7. Usuario de BD con permisos mínimos; nunca `root` en la app.
8. No mostrar errores internos de MySQL al usuario.
9. Usar `utf8mb4` para soportar tildes, ñ y emojis.
10. Hacer backups y probar la restauración.

Hash de contraseña

```
$hash = password_hash($clave, PASSWORD_DEFAULT);

if (password_verify($clave, $hash)) {
    // correcto
}
```

14. phpMyAdmin y consola

phpMyAdmin

- URL típica en XAMPP: `http://localhost/phpmyadmin`
- **Bases de datos** → crear base con `utf8mb4_unicode_ci`.
- **SQL**: ejecutar consultas sueltas o varias separadas por `;`.
- **Importar**: subir un `.sql` (backup o script de la cátedra).
- **Exportar**: generar `.sql` de estructura y/o datos.
- **Privilegios**: crear usuarios y asignar permisos.
- **Diseñador**: ver relaciones entre tablas.

Consola

```
mysql -u root -p
SHOW DATABASES;
USE pow;
SHOW TABLES;
DESCRIBE noticias;
SELECT * FROM noticias LIMIT 5;
SOURCE C:/ruta/script.sql;
```

Comandos de diagnóstico

```
SHOW PROCESSLIST;
SHOW STATUS;
SHOW VARIABLES LIKE 'version';
SELECT VERSION();
```

15. Plantillas para el parcial

15.1 Crear el esquema completo

```
DROP TABLE IF EXISTS noticia_etiqueta;
DROP TABLE IF EXISTS noticias;
DROP TABLE IF EXISTS etiquetas;
DROP TABLE IF EXISTS categorias;
DROP TABLE IF EXISTS usuarios;

CREATE TABLE usuarios (
  id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  usuario    VARCHAR(50) NOT NULL UNIQUE,
  clave_hash VARCHAR(255) NOT NULL,
  rol        ENUM('admin','editor','lector') NOT NULL DEFAULT 'lector'
) ENGINE=InnoDB;

CREATE TABLE categorias (
  id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE noticias (
  id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  titulo       VARCHAR(150) NOT NULL,
  cuerpo       TEXT NOT NULL,
  categoria_id INT UNSIGNED NOT NULL,
  autor_id     INT UNSIGNED,
  fecha        DATE NOT NULL,
  CONSTRAINT fk_noticia_categoria
    FOREIGN KEY (categoria_id) REFERENCES categorias(id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_noticia_autor
    FOREIGN KEY (autor_id) REFERENCES usuarios(id)
    ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE etiquetas (
  id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;
```

```
CREATE TABLE noticia_etiqueta (
  noticia_id  INT UNSIGNED NOT NULL,
  etiqueta_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (noticia_id, etiqueta_id),
  FOREIGN KEY (noticia_id)  REFERENCES noticias(id)  ON DELETE CASCADE,
  FOREIGN KEY (etiqueta_id) REFERENCES etiquetas(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

15.2 Listado con JOIN + GROUP BY

```
SELECT c.nombre AS categoria,
       COUNT(n.id) AS cantidad,
       MAX(n.fecha) AS ultima_fecha
FROM categorias c
LEFT JOIN noticias n ON n.categoria_id = c.id
GROUP BY c.id, c.nombre
ORDER BY cantidad DESC;
```

15.3 Búsqueda con filtros opcionales

```
SELECT n.id, n.titulo, c.nombre AS categoria, n.fecha
FROM noticias n
JOIN categorias c ON c.id = n.categoria_id
WHERE (? = '' OR n.titulo LIKE CONCAT('%', ?, '%'))
  AND (? = 0 OR n.categoria_id = ?)
ORDER BY n.fecha DESC;
```

15.4 Insertar desde un formulario

```
<?php
$titulo = trim($_POST['titulo'] ?? '');
$cuerpo = trim($_POST['cuerpo'] ?? '');
$cat    = (int) ($_POST['categoria_id'] ?? 0);
$fecha  = $_POST['fecha'] ?? date('Y-m-d');

if ($titulo === '' || $cuerpo === '' || $cat <= 0) {
    die('Faltan datos obligatorios');
}

$sql = "INSERT INTO noticias (titulo, cuerpo, categoria_id, fecha)
        VALUES (?, ?, ?, ?)";
$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'ssis', $titulo, $cuerpo, $cat, $fecha);
mysqli_stmt_execute($stmt);
mysqli_stmt_close($stmt);

header('Location: listado.php');
exit;
```

15.5 Actualizar y eliminar

```
// Actualizar
$sql = "UPDATE noticias SET titulo = ?, cuerpo = ? WHERE id = ?";
$stmt = mysqli_prepare($conexion, $sql);
mysqli_stmt_bind_param($stmt, 'ssi', $titulo, $cuerpo, $id);
mysqli_stmt_execute($stmt);

// Eliminar
$stmt = mysqli_prepare($conexion, "DELETE FROM noticias WHERE id = ?");
mysqli_stmt_bind_param($stmt, 'i', $id);
mysqli_stmt_execute($stmt);
```

15.6 Reporte con subconsulta

```
SELECT c.nombre,
       (SELECT COUNT(*) FROM noticias n WHERE n.categoria_id = c.id) AS total
FROM categorias c
ORDER BY total DESC;
```

15.7 Login + sesión (resumen)

```
session_start();

$stmt = mysqli_prepare($conexion,
```

```
    "SELECT id, clave_hash, rol FROM usuarios WHERE usuario = ? LIMIT 1");
mysqli_stmt_bind_param($stmt, 's', $usuario);
mysqli_stmt_execute($stmt);
$res = mysqli_stmt_get_result($stmt);
$u = mysqli_fetch_assoc($res);

if ($u && password_verify($clave, $u['clave_hash'])) {
    session_regenerate_id(true);
    $_SESSION['usuario_id'] = $u['id'];
    $_SESSION['rol'] = $u['rol'];
    header('Location: panel.php');
    exit;
}
```

---

16. Ejercicios resueltos

Ejercicio 1 — Crear la tabla `alumnos`

```
CREATE TABLE alumnos (
  id       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre   VARCHAR(80) NOT NULL,
  apellido VARCHAR(80) NOT NULL,
  email    VARCHAR(120) NOT NULL UNIQUE,
  edad     TINYINT UNSIGNED,
  nota     DECIMAL(4,2) DEFAULT 0,
  activo   BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;
```

Ejercicio 2 — Insertar tres alumnos

```
INSERT INTO alumnos (nombre, apellido, email, edad, nota) VALUES
('Ana',   'Gómez',  'ana@example.com',   22, 8.5),
('Luis',  'Pérez',  'luis@example.com',  25, 7.0),
('María', 'López',  'maria@example.com', 19, 9.25);
```

Ejercicio 3 — Alumnos mayores de 21, ordenados por apellido

```
SELECT nombre, apellido, edad
FROM alumnos
WHERE edad > 21
ORDER BY apellido ASC;
```

Ejercicio 4 — Cantidad de alumnos por estado

```
SELECT activo, COUNT(*) AS total
FROM alumnos
GROUP BY activo;
```

Ejercicio 5 — Promedio de notas por edad

```
SELECT edad, AVG(nota) AS promedio
FROM alumnos
GROUP BY edad
HAVING AVG(nota) >= 7
ORDER BY promedio DESC;
```

Ejercicio 6 — Alumnos sin email cargado

```
SELECT * FROM alumnos WHERE email IS NULL;
```

Ejercicio 7 — Actualizar la nota de un alumno

```
UPDATE alumnos
SET nota = 9.0
WHERE email = 'luis@example.com';
```

Ejercicio 8 — Eliminar alumnos inactivos

```
DELETE FROM alumnos WHERE activo = FALSE;
```

Ejercicio 9 — Crear una vista

```
CREATE OR REPLACE VIEW v_alumnos_aprobados AS
SELECT nombre, apellido, nota
FROM alumnos
WHERE nota >= 6;
```

Ejercicio 10 — Top 3 de notas

```
SELECT nombre, apellido, nota
FROM alumnos
ORDER BY nota DESC
LIMIT 3;
```

---

17. Apéndice: proyecto de convenios

Esquema mínimo sugerido a partir del relevamiento del proyecto (convenios marco/específicos, resoluciones, anexos, firmantes y usuarios).

```
CREATE TABLE usuarios (
  id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL,
  email      VARCHAR(120) NOT NULL UNIQUE,
  clave_hash VARCHAR(255) NOT NULL,
  rol        ENUM('admin','cargador','consulta') NOT NULL DEFAULT 'consulta',
  activo     BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE convenios (
  id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  numero            VARCHAR(30) NOT NULL UNIQUE,
  tipo              ENUM('marco','especifico') NOT NULL,
  titulo            VARCHAR(200) NOT NULL,
  descripcion       TEXT,
  texto_completo    LONGTEXT,
  archivo_imagen    VARCHAR(255),
  fecha_firma       DATE,
  fecha_vencimiento DATE,
  estado            ENUM('borrador','firmado','vigente','vencido','archivado')
                    NOT NULL DEFAULT 'borrador',
  creado_por        INT UNSIGNED,
  creado_en         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_convenio_usuario
    FOREIGN KEY (creado_por) REFERENCES usuarios(id)
    ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE resoluciones (
  id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  numero  VARCHAR(30) NOT NULL,
  anio    SMALLINT UNSIGNED NOT NULL,
  texto   LONGTEXT,
  archivo VARCHAR(255),
  fecha   DATE,
  UNIQUE KEY uq_resolucion (numero, anio)
) ENGINE=InnoDB;

CREATE TABLE anexos (
  id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  resolucion_id INT UNSIGNED NOT NULL,
  descripcion   VARCHAR(255),
  archivo       VARCHAR(255),
  CONSTRAINT fk_anexo_resolucion
    FOREIGN KEY (resolucion_id) REFERENCES resoluciones(id)
    ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE firmantes (
  id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  cargo  VARCHAR(100)
) ENGINE=InnoDB;
```

```
CREATE TABLE convenio_firmante (
  convenio_id INT UNSIGNED NOT NULL,
  firmante_id INT UNSIGNED NOT NULL,
  rol_firma   VARCHAR(50),
  fecha_firma DATE,
  PRIMARY KEY (convenio_id, firmante_id),
  FOREIGN KEY (convenio_id) REFERENCES convenios(id) ON DELETE CASCADE,
  FOREIGN KEY (firmante_id) REFERENCES firmantes(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE convenio_resolucion (
  convenio_id   INT UNSIGNED NOT NULL,
  resolucion_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (convenio_id, resolucion_id),
  FOREIGN KEY (convenio_id)   REFERENCES convenios(id)    ON DELETE CASCADE,
  FOREIGN KEY (resolucion_id) REFERENCES resoluciones(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Búsqueda de texto completo

```
ALTER TABLE convenios
  ADD FULLTEXT INDEX ft_convenios (titulo, descripcion, texto_completo);

SELECT id, numero, titulo
FROM convenios
WHERE MATCH(titulo, descripcion, texto_completo)
      AGAINST (? IN NATURAL LANGUAGE MODE);
```

Alternativa con `LIKE` (más simple, más lenta en tablas grandes):

```
SELECT id, numero, titulo
FROM convenios
WHERE titulo LIKE CONCAT('%', ?, '%')
   OR descripcion LIKE CONCAT('%', ?, '%')
   OR texto_completo LIKE CONCAT('%', ?, '%');
```

Convenios por vencer en los próximos 30 días

```
SELECT numero, titulo, fecha_vencimiento,
       DATEDIFF(fecha_vencimiento, CURDATE()) AS dias_restantes
FROM convenios
WHERE estado = 'vigente'
  AND fecha_vencimiento BETWEEN CURDATE()
                            AND DATE_ADD(CURDATE(), INTERVAL 30 DAY)
ORDER BY fecha_vencimiento ASC;
```

Convenios con sus resoluciones y cantidad de anexos

```
SELECT c.numero,
       c.titulo,
       GROUP_CONCAT(DISTINCT r.numero ORDER BY r.anio SEPARATOR ', ') AS resoluciones,
       COUNT(DISTINCT a.id) AS anexos
FROM convenios c
LEFT JOIN convenio_resolucion cr ON cr.convenio_id = c.id
LEFT JOIN resoluciones r       ON r.id = cr.resolucion_id
LEFT JOIN anexos a             ON a.resolucion_id = r.id
GROUP BY c.id, c.numero, c.titulo
ORDER BY c.numero;
```

Convenios cargados por usuario

```
SELECT u.nombre, COUNT(c.id) AS total_convenios
FROM usuarios u
LEFT JOIN convenios c ON c.creado_por = u.id
GROUP BY u.id, u.nombre
ORDER BY total_convenios DESC;
```

Convenios marco y sus específicos

```
SELECT marco.numero AS convenio_marco,
       espec.numero AS convenio_especifico,
       espec.titulo
FROM convenios marco
JOIN convenios espec
```

```
  ON espec.tipo = 'especifico'
 AND espec.numero LIKE CONCAT(marco.numero, '%')
WHERE marco.tipo = 'marco';
```

> Si la relación marco–específico se modela explícitamente, conviene agregar `convenio_marco_id` en `convenios` con FK a `convenios(id)`. Evita depender del formato del número.

---

18. Errores típicos y checklist final

Errores frecuentes

Error	Causa probable	Solución
`Unknown database 'pow'`	La base no existe.	`CREATE DATABASE pow;`
`Table doesn't exist`	No se seleccionó la base o el nombre está mal.	`USE pow;` / revisar `SHOW TABLES;`
`Cannot add or update a child row: a foreign key constraint fails`	La FK apunta a un ID inexistente.	Insertar primero el padre o corregir el ID.
`Cannot delete or update a parent row`	Hay hijos con `RESTRICT`.	Borrar hijos primero o usar `ON DELETE CASCADE`.
`Column count doesn't match value count`	`INSERT` con distinta cantidad de columnas y valores.	Revisar la lista.
`Duplicate entry`	Se violó `UNIQUE`/PK.	Usar otro valor o `ON DUPLICATE KEY UPDATE`.
`You have an error in your SQL syntax`	Coma, comilla o palabra reservada.	Revisar; usar backticks si hace falta.
`Access denied for user`	Usuario/clave/permisos.	Revisar `GRANT` y credenciales.
`Data too long for column`	Texto más largo que el `VARCHAR`.	Ampliar columna o validar.
`Incorrect date value`	Formato de fecha inválido.	Usar `YYYY-MM-DD`.
`mysqli_stmt_get_result()` undefined	Falta mysqlnd.	Usar `bind_result()` / `fetch()` o habilitar mysqlnd.
`Headers already sent`	`header()` después de imprimir.	Poner `header()` antes de cualquier salida.

Checklist antes de entregar

- [ ] La base y las tablas se crean sin errores desde cero.
- [ ] Las PK están definidas y las FK tienen el tipo correcto (`INT UNSIGNED`).
- [ ] `ON DELETE` elegido conscientemente (`RESTRICT` por defecto).
- [ ] `utf8mb4` para textos con tildes/ñ.
- [ ] Las consultas usan `?` y `bind_param` si reciben datos del usuario.
- [ ] Las salidas usan `htmlspecialchars()`.
- [ ] Las claves se guardan con `password_hash()`.
- [ ] El `WHERE` de `UPDATE`/`DELETE` está probado con un `SELECT`.
- [ ] Las consultas con `GROUP BY` incluyen todas las columnas no agregadas.
- [ ] Se probó el login, el alta, la edición, el borrado y la paginación.
- [ ] Hay un backup `.sql` de la base.
- [ ] No se conecta como `root` desde la aplicación.

Frases útiles para justificar

- "La FK garantiza la integridad referencial: no se puede cargar una noticia con una categoría inexistente."
- "La tabla intermedia resuelve la relación N:M y su PK compuesta evita duplicados."
- "El índice acelera la búsqueda por `fecha`, a costa de un poco de espacio y de velocidad de escritura."
- "La transacción asegura atomicidad: o se descuentan y acreditan ambos saldos, o no se hace ninguno."
- "El prepared statement separa el SQL de los datos, por lo que impide la inyección SQL."
- "Normalizar reduce redundancia y evita anomalías de actualización."

Fin del cheatsheet.
Si tenés que llevar una sola hoja al parcial, priorizá: DDL con PK/FK, `SELECT` con `JOIN` + `GROUP BY` + `HAVING`, subconsultas,

`INSERT/UPDATE/DELETE` con prepared statements, paginación y login con sesión.